

Module Title:	Concurrent Programming
Module Code:	6SENG002W
In-Class Test:	17th January, 2022
Start Time:	11:00
Submission Deadline:	13:15
RAF Submission Deadline:	13:50

INSTRUCTIONS FOR CANDIDATES

There are EIGHT questions in the test.

Answer ALL EIGHT questions.

Questions 1 - 4 are worth 10 marks each.

Questions 5 - 8 are worth 15 marks each.

**YOU MUST SUBMIT YOUR ANSWERS BEFORE THE
SUBMISSION DEADLINE.**

Question 1

Explain what each of the following concurrency concepts mean:

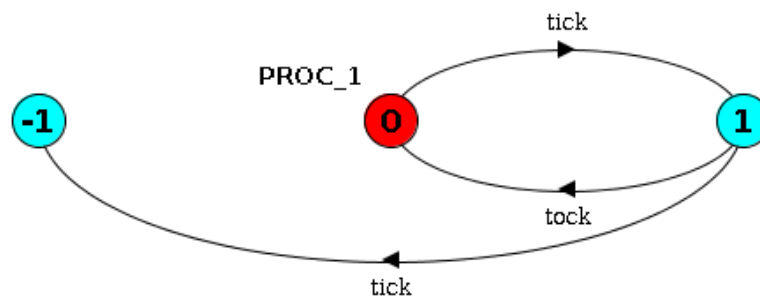
- (a) race conditions
- (b) synchronisation
- (c) interleaving
- (d) mutual exclusion
- (e) deadlock

[10 marks]
[TOTAL 10]

Question 2

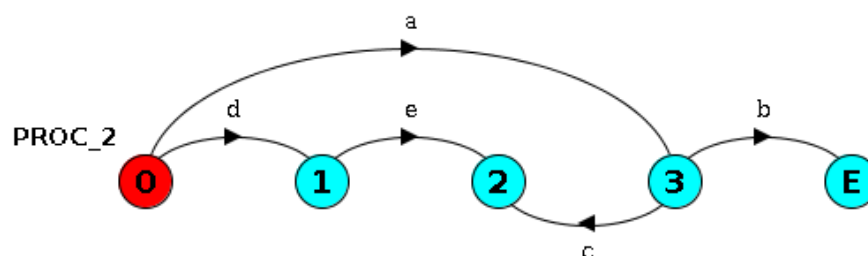
For the following two *Labelled Transition System (LTS)* graphs give the corresponding FSP process definitions.

(a)



[4 marks]

(b)



[6 marks]
[TOTAL 10]

Question 3

- (a) Briefly describe the different ways to create a Java thread. [4 marks]
 - (b) When a user defines their own thread class, a constructor method is sometimes used, what should always be called from within the constructor? [2 marks]
 - (c) How do you define the body of a thread? [2 marks]
 - (d) How do you initiate thread execution? [2 marks]
- [TOTAL 10]

Question 4

- (a) Describe the main features of the monitor concurrent programming language mechanism that is used to ensure safe resource sharing in a concurrent program. [6 marks]
 - (b) In a Java version of a monitor, the monitor methods often include a `while` loop. What is the purpose of these `while` loops? [4 marks]
- [TOTAL 10]

Question 5

Given the following FSP system.

```
const N      = 3
range DATA = 1..N
```

```
Producer = ( produce[2] -> in[2] -> Producer ) .
```

```
BUFFER   = ( in[ i : DATA ] -> out[ i ] -> BUFFER ) .
```

```
Consumer = ( out[ x : DATA ] -> consume[x] -> Consumer ) .
```

```
||SYSTEM = ( Producer || BUFFER || Consumer ) .
```

(a) Draw the alphabet diagram for SYSTEM.

[5 marks]

(b) For each action state:

- the type of action: synchronous or asynchronous.
- all the processes that perform it.

[10 marks]

[TOTAL 15]

Question 6

(a) Draw a diagram that illustrates the relationships between the life-cycle states of a Java thread.

[6 marks]

(b) Give a brief description of each state.

[9 marks]

[TOTAL 15]

Question 7

With reference to the Java code for the Producer Consumer Problem given in Appendix A, answer the following questions.

(a) Explain why the Buffer class (lines 1 – 34) is a “secure” *monitor*. [4 marks]

(b) In the Buffer’s put method, what is the purpose of the while-loop (lines: 21 – 26)? What is the effect of the while-loop? [4 marks]

(c) Assume that the Buffer’s new_data variable is false, the Producer thread p1 is “sleeping”, and the Consumer thread c1 starts to execute line 70:

```
70          int data = buffer.take() ;
```

What then happens to the Consumer thread c1? [4 marks]

(d) In the definition of the Buffer’s take method (lines 6 – 17), which lines of the method does the monitor’s synchronisation lock change its status, e.g. from locked to unlocked, or from unlocked to locked? Further, state which lock status change or changes happen on those lines. [3 marks]

[TOTAL 15]

Question 8

(a) Describe the features of the *semaphore* concurrent programming mechanism. [5 marks]

(b) What is the *Dining Philosophers* problem (for 5 Philosophers)? Explain how *deadlock* can occur and how it can be avoided by the use of a *Butler*. [5 marks]

(c) Explain what semaphores would be needed to construct a deadlock free solution to the Dining Philosophers problem using a Butler, and how they would be used. [5 marks]

[TOTAL 15]

Appendix A

This appendix contains the Java code for a simple version of the Producer Consumer problem.

```
1    class Buffer
2    {
3        private int contents = -1 ;
4        private boolean new_data = false ;
5
6        public synchronized int take( )
7        {
8            while ( !new_data )
9            {
10                try {
11                    wait() ;
12                } catch(InterruptedException e){ }
13            }
14            new_data = false ;
15            notifyAll() ;
16            return contents ;
17        }
18
19        public synchronized void put( int value )
20        {
21            while ( new_data )
22            {
23                try {
24                    wait() ;
25                } catch(InterruptedException e){ }
26            }
27
28            contents = value ;
29            new_data = true ;
30
31            notifyAll() ;
32        }
33
34    } // Buffer
```

```
35  class Producer extends Thread
36  {
37      private final Buffer buffer ;
38
39      public Producer( Buffer buffer )
40      {
41          super( "Producer" ) ;
42          this.buffer = buffer ;
43      }
44
45      public void run()
46      {
47          for (int i = 0; i < 10; i++)
48          {
49              buffer.put( i ) ;
50              System.out.println(getName() + " put:  " + i) ;
51              try { sleep( 1000 ) ; }
52              catch (InterruptedException e) { }
53          }
54      }
55  } // Producer

56  class Consumer extends Thread
57  {
58      private final Buffer buffer ;
59
60      public Consumer( Buffer buffer )
61      {
62          super( "Consumer" ) ;
63          this.buffer = buffer ;
64      }
65
66      public void run()
67      {
68          for (int i = 0; i < 10; i++)
69          {
70              int data = buffer.take() ;
71              System.out.println(getName() + " taken: " + data) ;
72          }
73      }
74  } // Consumer
```

```
75  class ProducerConsumerProblem
76  {
77      public static void main( String args[] )
78      {
79          Buffer buffer = new Buffer() ;
80
81          Producer p1 = new Producer( buffer ) ;
82          Consumer c1 = new Consumer( buffer ) ;
83
84          p1.start() ;
85          c1.start() ;
86      }
87  } // ProducerConsumerProblem
```

END OF THE IN-CLASS TEST PAPER